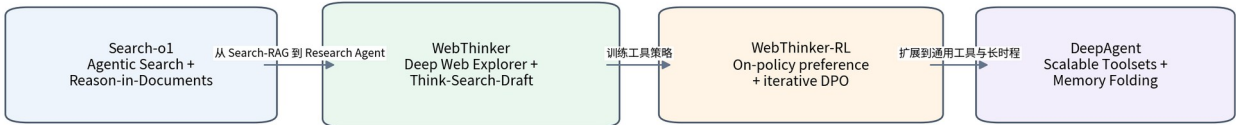


WebThinker

完整研究：Reasoning-Native Deep Research

Deep Web Explorer、Autonomous Think-Search-and-Draft、Online DPO 与源码工程解
析

RUC-NLPIR Deep Research 演化：Search-o1 → WebThinker → DeepAgent



研究基线：WebThinker NeurIPS 2025 论文（arXiv:2504.21776 v2）、RUC-NLPIR/WebThinker 官方仓库与 README、当前开源实现。研究日期：2026-08-13。

核心判断

WebThinker 的关键创新不是“给推理模型加一个 Search Tool”，而是把 Search / Click / Read / Draft / Check / Edit 变成**长推理轨迹中的原生动作**，并通过嵌套 Explorer 与在线偏好优化，让模型学习何时获取外部知识、何时继续深挖、何时写作和停止。

目录与执行摘要

1. 为什么 WebThinker 出现：从 RAG Workflow 到 Reasoning Agent
2. 总体架构与两种工作模式
3. Deep Web Explorer：嵌套搜索与页面导航
4. Autonomous Think-Search-and-Draft
5. Document Memory 与 Auxiliary LLM
6. Tool Protocol：Special Tokens + Host Runtime
7. Online DPO：如何训练 Research Tool Policy
8. 训练数据与 Preference Pair 构造
9. 实验体系：GPQA / GAIA / WebWalkerQA / HLE / Glaive
10. 复杂问题求解结果
11. 报告生成结果
12. Ablation 与论文内部数据不一致
13. 跨 LRM Backbones 的适应性
14. 源码级实现：Problem Solving Mode
15. 源码级实现：Report Generation Mode
16. 工程可靠性与生产化缺口
17. WebThinker vs Search-o1 / MindSearch / STORM
18. WebThinker vs Open Deep Research / GPT Researcher
19. WebThinker vs Tongyi / DeerFlow
20. 可复用 Know-why / Know-how
21. 从零实现 WebThinker-style MVP
22. 面向 2026 的下一代改进
23. 来源与证据等级

一句话

MindSearch 把“搜索空间”图化；WebThinker 更进一步，把“搜索动作”塞进推理轨迹本身。其目标不是更复杂的 Workflow，而是让模型把 Web Research 当作思考的一部分。

WebThinker 总体架构：主 LRM 负责策略，Explorer 负责深搜，Aux LLM 负责文本操作

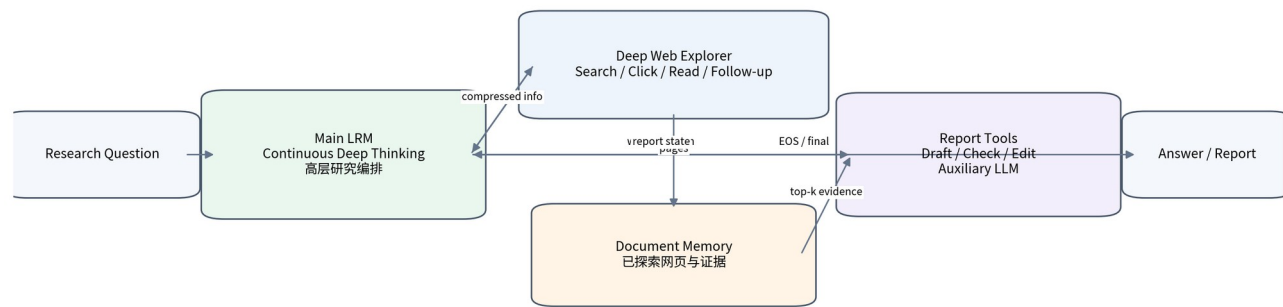


图 1 WebThinker 的角色分工：Main LRM = 策略；Explorer = 深搜；Aux LLM = 阅读与写作执行

1. 为什么 WebThinker 出现：从 RAG Workflow 到 Reasoning Agent

WebThinker 的论文把问题定义得非常明确：LRM 已经能做长链推理，但内部知识是静态的；传统 RAG 能带来外部知识，却通常把搜索固定在**预定义 workflow**中。于是系统会出现一个错位：推理模型在动态思考，检索系统却仍按固定阶段工作。

作者把三种范式放在同一张图中：Standard RAG 是“一次搜索 → 一次回答”；Advanced / Iterative RAG 是“Planner → 多次 Search → Merge → 判断是否继续”；WebThinker 则把工具调用嵌入 continuous deep-thinking，模型在推理 token 流中自行决定何时调用工具。论文把这一点表述为 end-to-end task execution in a single generation。

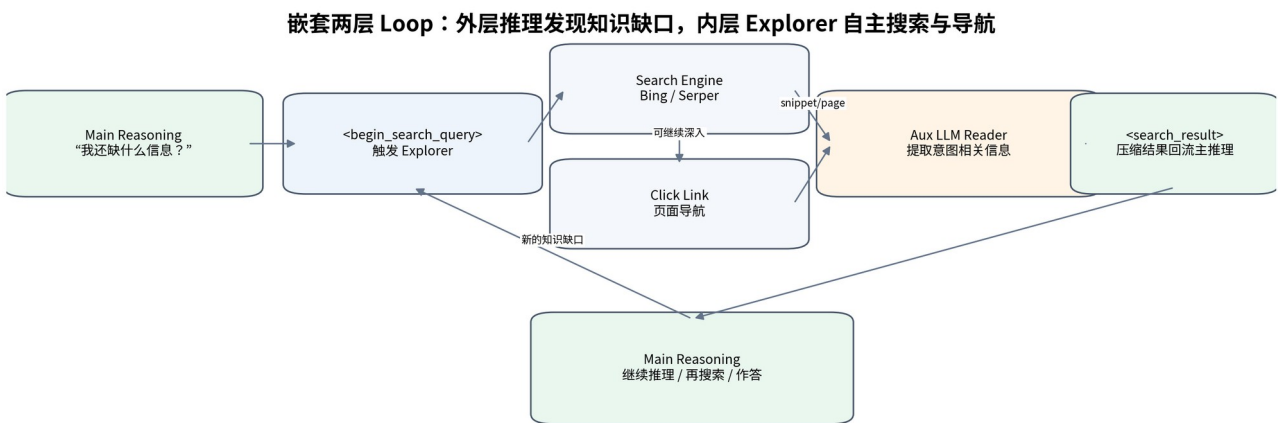


图 2 WebThinker 的真正核心：外层 reasoning loop + 内层 explorer loop

Know-why

如果“何时搜索、搜什么、要不要点开网页、什么时候信息足够”仍由固定 Workflow 决定，那么 LRM 的长程推理能力无法真正作用于信息获取策略。WebThinker 的设计目标，是把**信息获取决策本身**纳入 reasoning policy。

2. 总体架构与两种工作模式

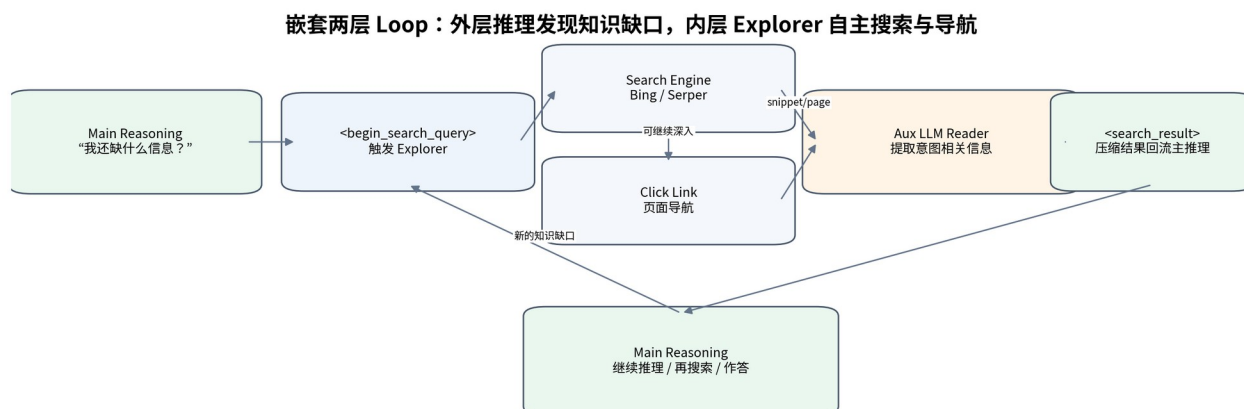
WebThinker 有两种模式。**Problem-Solving Mode** 面向复杂 QA：主 LRM 在推理中调用 Deep Web Explorer，Explorer 深度搜索并返回与当前知识缺口相关的信息。**Report Generation Mode** 在此基础上增加 Draft / Check / Edit，让报告内容随研究进度逐步生长，而不是最后把全部检索结果一次性塞进 writer。

模式	主目标	主 LRM 动作	外部执行	终止
Problem Solving	解决复杂知识密集问题	Think ↔ Search	Explorer: search/click/read	生成最终答案
Report Generation	形成完整研究报告	Think ↔ Search ↔ Draft ↔ Check ↔ Edit	Explorer + Aux Writer + Document Memory	EOS / 完成报告

3. Deep Web Explorer：嵌套搜索与页面导航

Deep Web Explorer 不是普通 search API wrapper。它收到外层 LRM 产生的信息需求后，自己再运行一个小型 Agent Loop：生成 search query、查看搜索结果、决定是否继续搜索、决定点击哪一个 URL、读取页面、根据页面内容判断是否继续深入，最后输出一个简洁的信息块返回主推理链。

论文形式化为两层概率过程：外层主 reasoning chain 使用 Explorer 的输出作为历史 observation；内层 Explorer 的 reasoning chain 又受到当前搜索 query、网页集合与动态页面内容的条件约束。**这等价于把“Research”封装成一个有内部策略的工具。**



3.1 源码里的 Explorer 保护机制

官方 `run\_web\_thinker.py` 显示，Explorer 不是无限循环。当前代码设置 `MAX\_INTERACTIONS = 10`，并维护 `clicked\_urls` 与 `executed\_search\_queries` 防止重复动作；网页抓取失败时使用 error indicator 列表判断错误页面，并优先尝试其他链接。搜索结果与 URL 内容都有 cache。

```
MAX_INTERACTIONS = 10
clicked_urls = set()
executed_search_queries = set()
...
if new_query in executed_search_queries:
    return "already searched"
if url in clicked_urls:
    return "already clicked"
```

值得注意的是，开源代码里对 token 数的部分控制使用 `len(text.split())` 做近似。这足以作为研究原型，但生产环境应换成模型 tokenizer 或服务端 usage 计数，否则在中英混合、代码、URL 场景下误差会很大。

3.2 Search 与 Read 被明确分开

Search Engine 负责发现候选页面，Click + Web Page Reader 负责把网页变成“当前 intent 可用的信息”。这与成熟 Research Agent 的通用原则一致：**Search = discovery; Read = evidence acquisition**。如果只用 snippet，系统会停留在浅层信息检索，无法实现论文所强调的 deep web exploration。

4. Autonomous Think-Search-and-Draft

WebThinker 最值得研究的第二个设计，是把写作操作纳入 agent action space。传统 Deep Research 往往是 Research Phase 完成后再进入 Writing Phase；WebThinker 则允许主 LRM 在任何时刻决定：某个章节证据已经足够，可以先写；之后又发现新信息时，再检查和编辑。

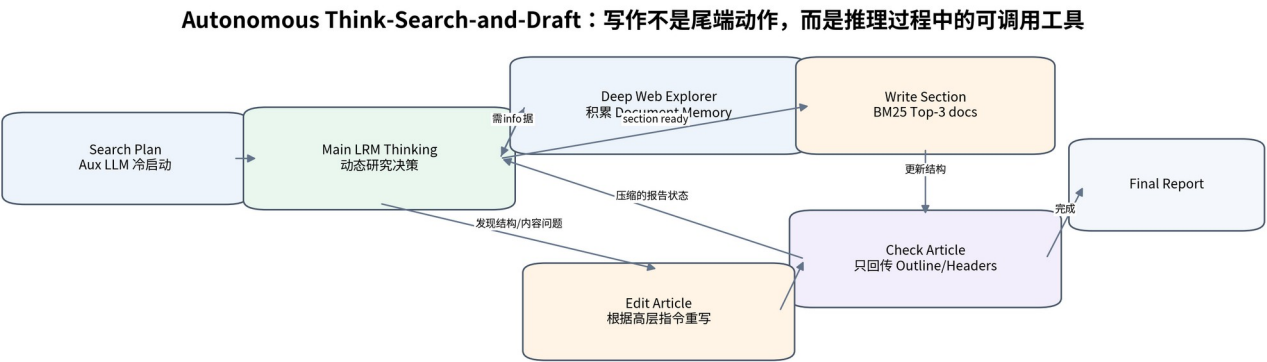


图 3 Think-Search-and-Draft 把报告视为 “可持续更新的外部状态”

三个核心写作动作是：**Draft** 写一个章节，**Check** 查看当前报告状态，**Edit** 根据高层编辑指令修改报告。这里的关键不是工具数量，而是把 report state 从 “最后一次输出” 变成运行期 artifact。

4.1 为什么不能让主 LRM 自己直接写所有文本

论文与源码都采用 Main LRM + Auxiliary LLM 分工：主 LRM 做高价值、高不确定性的研究决策；Auxiliary LLM 做页面阅读、search/click intent 提取、section drafting、editing 等较确定的文本操作。这样可以把昂贵 reasoning model 的 token 用在策略，而不是大段重复的文字变换。

Know-how

强模型做 policy，便宜/快模型做 execution，是 WebThinker 最可迁移的成本工程原则之一。可抽象为：
`Strong LRM = Controller`，`Aux LLM = Text Operator`，`Host Runtime = Deterministic Executor`。

5. Document Memory 与 Auxiliary LLM

报告模式会把探索到的网页持续写入 `document\_memory`。在写章节时，开源实现使用 BM25 对 document memory 检索，取 Top-3 相关文档交给辅助模型。主 LRM 不直接吞所有页面正文，只持有研究轨迹与一个压缩的 report status。

这是非常典型的 Context Engineering：**外部化 Evidence State，主上下文只保留决策所需状态**。当前源码甚至在 Check Article 时只把文章标题与各级 header 拼接回主推理上下文，而不是把完整长文再次复制进去。

```
headers = re.findall(r'^#{1,4}\s+.*$', article, re.MULTILINE)
summarized_article = '\n'.join(headers) + '\n'
```

Know-why

报告越长，越不能让 Planner 每一轮都重读全文。WebThinker 用“Outline/Headers 作为报告状态摘要”的做法，本质上是在做 artifact-state compression。

6. Tool Protocol : Special Tokens + Host Runtime

开源实现没有依赖通用 function-calling schema，而是定义一组明确的特殊

marker: `begin\_search\_query/end\_search\_query`、`begin\_click\_link/end\_click\_link`；报告模式进一步加入 `begin\_write\_section`、`begin\_edit\_article`、`begin\_check\_article`。Host 检测模型输出结尾，然后执行对应动作。

源码协议：模型只生成动作 marker，Host 负责确定性执行与结果注入

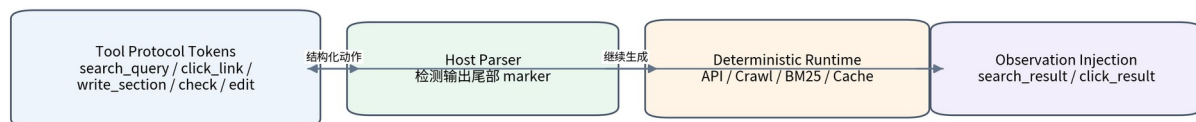


图4 模型生成动作，Runtime 执行动作：概率性决策与确定性约束分离

这种设计的优势是协议非常透明、可在开源推理模型上工作、易于采集完整 trajectory；缺点是 parser 对格式错误敏感，需要做 marker 验证、重复检测、错误 recovery。2026 年的生产实现更适合把同样思想迁移到 typed tool calls / grammar-constrained decoding。

6.1 Report Mode 的冷启动 Search Plan

源码中 Report Generation 并非完全“零计划”。开始时由 Auxiliary LLM 先生成 `search\_plan`，再把 plan 注入主 LRM 的 report instruction。随后真正的研究、搜索和写作动作仍由主 LRM 动态决定。也就是说，它是 **weak upfront plan + strong online replanning**。

7. Online DPO : 如何训练 Research Tool Policy

WebThinker 的 RL 部分并不是用一个复杂 reward model 给每一步 tool call 打分，而是通过完整研究轨迹之间的偏好排序训练模型。对同一 query 自采样多条 trajectory，然后按层级规则构造 preferred / dispreferred pair。

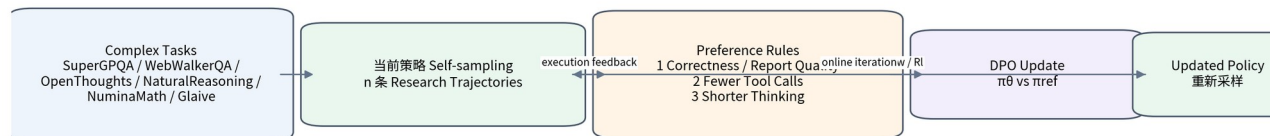


图5 WebThinker 的训练闭环：当前策略产生数据，execution feedback 再反过来更新策略

8. 训练数据与 Preference Pair 构造

论文用于 self-sampling 的任务来源包括 SuperGPQA、WebWalkerQA、OpenThoughts、NaturalReasoning、NuminaMath 与 Glaive，覆盖科学问答、网页导航、通用推理、数学以及开放式报告。每个 query 由当前 LRM 采样 n 条不同轨迹，轨迹包含主推理链与 Explorer 内部 tool usage。

优先级	Preference 规则	为什么重要
1	最终正确性 / 报告质量更高	首先保证 Research 是有效的
2	两者都正确时，tool calls 更少者胜	把搜索成本纳入策略优化
3	正确且工具数相同，显著更短的 thinking 胜	惩罚无效长思考与重复推理

随后执行 iterative online DPO：用当前 preference set 更新 policy；新 policy 再重新采样；重新构造 preference pairs；把新 policy 变成下一轮 reference policy。论文实验使用 **2 iterations online DPO**、训练 max sequence length 32,768。

Know-how

Research Agent 的训练目标不应该只有 answer accuracy。WebThinker 把 ****Correctness × Tool Efficiency × Reasoning Efficiency**** 组成层级偏好，这是一种非常工程化的 trajectory-level reward design。

9. 实验体系

论文把评测分成两类。复杂问题求解使用 GPQA（PhD-level science）、GAIA（复杂通用助手任务）、WebWalkerQA（深层网页导航）与 HLE（极难跨学科题）。报告生成使用 Glaive 的开放式 research questions，维度包括 Completeness、Thoroughness、Factuality、Coherence。

主实验 backbone 为 QwQ-32B；Auxiliary model 使用同参数规模的 Qwen2.5-Instruct。推理参数包括 max generation 81,920 tokens、temperature 0.7、top_p 0.8、top_k 20、repetition penalty 1.05。论文当时使用 Bing Search (k=10) + Crawl4AI；当前官方 README 已加入 Google Serper，并明确提醒 Bing Search API 在 2025-08 退役。

10. 复杂问题求解结果

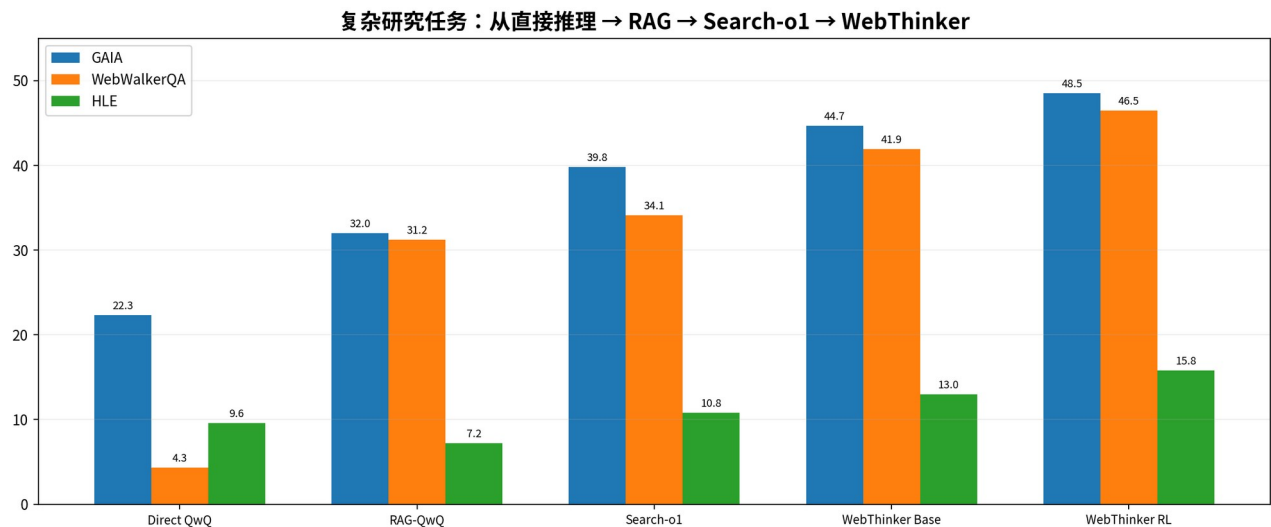


图 6 三类外部知识任务都显示：把搜索嵌入 reasoning trajectory 的收益高于普通 RAG

方法	GPQA Avg	GAIA Avg	WebWalker Avg	HLE Avg
QwQ-32B Direct	64.1	22.3	4.3	9.6
RAG-QwQ-32B	64.6	32.0	31.2	7.2
Search-o1-32B	67.2	39.8	34.1	10.8
WebThinker-32B-Base	68.7	44.7	41.9	13.0
WebThinker-32B-RL	70.7	48.5	46.5	15.8
OpenAI Deep Research (official release values)	-	67.4	-	26.6

这里最值得注意的不是绝对分数，而是 **Base 版本已经明显超过 Search-o1**。这说明仅靠 Deep Web Explorer 与更深的 search/click/read structure，就能带来较大增益；RL 在此基础上进一步优化工具策略。

论文在引言中强调相对 Search-o1，WebThinker 在 GAIA 与 HLE 的提升分别为 21.9% 与 36.2%（作者按相对提升口径表述）。从表格绝对值看，GAIA 39.8 → 48.5，HLE 10.8 → 15.8。

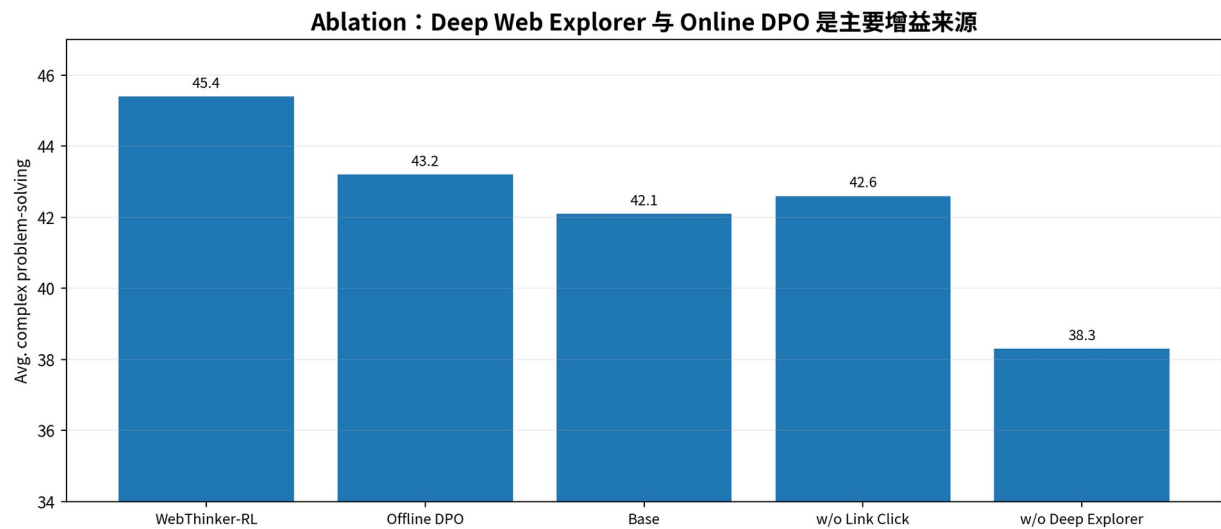
11. 报告生成结果

方法	Completeness	Thoroughness	Factuality	Coherence	Avg
RAG-Qwen2.5-72B	5.7	5.3	6.4	6.3	5.9
RAG-DeepSeek-R1	6.6	6.4	7.1	7.1	6.8
Grok3 DeeperSearch	6.4	6.1	7.0	6.5	6.5
Gemini2.0 Deep Research	8.1	8.0	7.7	7.7	7.9
WebThinker-32B- Base	8.4	8.2	7.7	7.8	8.0
WebThinker-32B- RL	8.3	8.4	7.7	7.9	8.1

报告任务有一个非常重要的结论：**Base 和 RL 差距很小**。这表明 Think-Search-and-Draft 的系统结构本身已经能解决大部分报告生成问题；RL 的主要价值更集中在复杂问题求解与工具效率，而不是单纯提升写作分数。

论文还用 t-SNE 做定性分析，认为 WebThinker 的报告在相同主题中形成更宽的内容子簇，说明它会根据写作进度继续搜索不同方向，而不是按固定 RAG prompt 一次性覆盖。这个结论属于论文作者的分析，应视为定性证据而非严格因果证明。

12. Ablation 与论文内部数据不一致



设置	Complex Avg	Report Avg	含义
WebThinker-32B-RL	45.4	8.1	完整系统
w/ Offline DPO	43.2	-	on-policy 更新优于一次性离线偏好训练
w/o Training (Base)	42.1	8.0	结构本身已很强
w/o Link Clicking	42.6	-	深层页面导航贡献明显
w/o Deep Web Explorer	38.3	7.7	最核心组件之一
w/o Report Check & Edit	-	7.7	Coherence 降低
w/o Auto. Report Draft	-	6.6	报告质量下降最大

证据审计

论文正文 4.7 的自然语言写成“RL Avg. 44.9”，但 Table 3 中 GPQA 70.7、GAIA 48.5、WebWalker 46.5、HLE 15.8 的平均值实际约为 **45.4**，表格也标为 45.4。这里存在论文内部不一致。本报告以 Table 3 与可复算数值为准，并保留该冲突。

这个细节很值得工程团队学习：Deep Research 的评测往往依赖多数据集、多 judge、多聚合指标，**结果表应做到可复算**。不要只复制论文摘要中的百分比。

13. 跨 LRM Backbones 的适应性

论文把 WebThinker 迁移到 DeepSeek-R1 Distill 7B / 14B / 32B。为了让较小模型稳定使用工具，作者先用 QwQ-32B WebThinker 产生的 **7.8k trajectories** 做 cold-start SFT，再做 RL。

论文报告 R1-7B 相对 direct generation 在 GAIA 与 WebWalkerQA 上分别提升 174.4% 与 422.6%，相对标准 RAG 分别提升 82.9% 与 161.3%。可迁移性说明工具协议与训练数据可以成为 “Research Policy Distillation” 载体。

Know-how

小模型直接上复杂工具环境容易出现格式错误和行为不稳定。先用强模型 trajectory 做 cold-start SFT，再做 on-policy 优化，是比 “直接 RL” 更稳健的工程路径。

14. 源码级实现：Problem Solving Mode

官方实现的核心是 `scripts/run\_web\_thinker.py`。主模型通过 `<|begin\_search\_query|>...<|end\_search\_query|>` 发出搜索动作。Host 解析 query、调用 Bing/Serper、抓取网页、生成 search intent，再启动 `generate\_deep\_web\_explorer()`。Explorer 最终信息通过 `<|begin\_search\_result|>` 注入回主模型 prompt，模型继续 completion。

```
while not seq["finished"]:
    if not output.endswith(END_SEARCH_QUERY): break
    search_query = extract_between(...)
    results = search(search_query)
    analysis = generate_deep_web_explorer(...)
    extracted_info = extract_answer_fn(analysis, mode="summary")
    prompt += BEGIN_SEARCH_RESULT + extracted_info + END_SEARCH_RESULT
    response = model(prompt, generate_mode="completion")
```

这使主推理链保持连续：模型不是每次 tool call 后重新从一个高层 workflow node 开始，而是在同一条生成轨迹上继续。

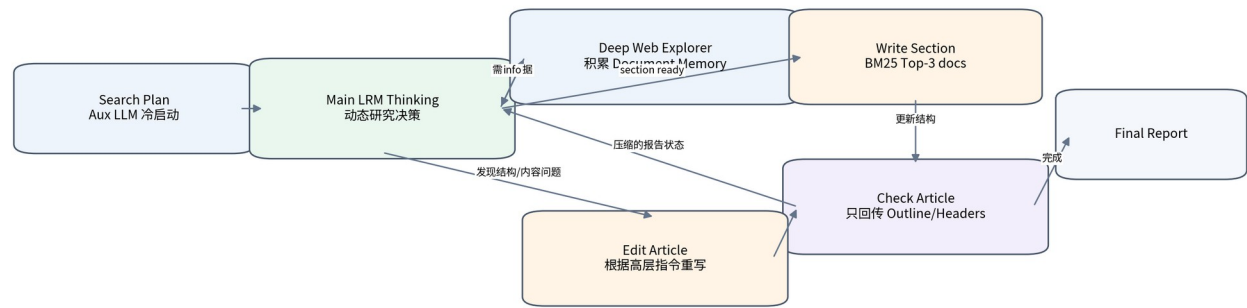
14.1 工程防护

- **重复查询/URL 去重**：维护 executed_search_queries 与 clicked_urls。
- **Search cache / URL cache**：减少 API 成本和重复抓取。
- **Fetch error detection**：对 JS、cookie、timeout、quota 等错误文本做 heuristics。
- **Interaction budgets**：外层 max_search_limit，Explorer 内部 MAX_INTERACTIONS。
- **超限强制收束**：达到上限后注入 “基于已收集信息给出 final information” 的提示。
- **Async API**：使用 AsyncOpenAI、aiohttp 与 semaphore 控制并发。

15. 源码级实现：Report Generation Mode

`scripts/run\_web\_thinker\_report.py` 在 Problem Solving Loop 上扩展三个 writing marker，并维护 `article`、`summarized\_article`、`document\_memory`。它的实现非常能体现 “报告是外部 artifact” 的思想。

Autonomous Think-Search-and-Draft：写作不是尾端动作，而是推理过程中的可调工具



写章节时，Runtime 构造 `section\_name + task` 的 BM25 query，从 document memory 中取 Top-3 相关文档给 Auxiliary LLM；写完后把完整文本存进 article，但只把 Markdown headers 形成的 summarized_article 回传给主 LRM。

Edit Article 时，主 LRM 只给高层 edit instruction，Auxiliary LLM 接收完整 article 并负责具体修改；Check Article 则主要把结构摘要回流给主 reasoning。这是典型的“**planner 看摘要，worker 看全文**”上下文分层。

16. 工程可靠性与生产化缺口

WebThinker 是非常有价值的研究型开源实现，但距离稳定生产系统仍有若干明显缺口。

问题	当前研究实现	生产化建议
Token budget	部分使用 split() 估算	统一 tokenizer / provider usage accounting
Tool protocol	特殊字符串 marker	typed schema / constrained decoding / validation
Page trust	抓取后交给 LLM 阅读	内容隔离、prompt injection 防护、source trust scoring
Citation	报告主要依赖文档文本	claim-level provenance + citation validator
Memory retrieval	BM25 Top-3	hybrid retrieval + rerank + source diversity
Stop policy	硬 interaction/token 上限	coverage / marginal information gain / confidence / cost
Failures	heuristic string indicators	typed exception taxonomy + retries + fallback parser
Observability	JSON trajectory output	trace/span/cost/tool success metrics

尤其需要强调：WebThinker 的重点是 research capability，不是完整的 evidence governance。一个生产 Deep Research 系统还必须解决网页 prompt injection、来源可信度、冲突证据、引用可验证性、robots/许可、动态页面重放等问题。

17. WebThinker vs Search-o1 / MindSearch / STORM

WebThinker 在 Deep Research 范式中的位置

系统	研究策略位置	搜索拓扑	Context 机制	是否训练 Agent	核心强项
STORM	Workflow	多视角问答 + Outline	Reference pool / section context	否	覆盖与长文结构
MindSearch	Planner Agent	Dynamic DAG	Searcher isolation + summary	否	搜索空间图化 / 并行
Open Deep Research	Supervisor	动态 subagent delegation	Subagent compression	否	通用 tool-calling
WebThinker	Reasoning trajectory 内	Nested Search/Click loop	Explorer output + doc memory	是：online DPO	搜索与推理深度耦合
Tongyi DeepResearch	模型策略内化	Native ReAct / IterResearch	训练出的长程研究状态	是：CPT/SFT/RL	端到端 Agent 能力
DeerFlow 2.0	Harness + Skills	动态工具/子代理	Filesystem / summary / memory	否	长时程运行时与工程治理

Search-o1 的核心是 search-enhanced reasoning：在推理中动态检索，并用 Reason-in-Documents 对返回文本进一步推理。WebThinker 在此基础上把网页导航与报告写作也纳入动作空间。

MindSearch 主要解决“应该搜索哪些子问题，以及依赖关系是什么”，强调 Dynamic DAG 与多 WebSearcher 并行；WebThinker 更关心“在当前 reasoning state 下，下一步是否需要 search/click/draft”，是 trajectory-centric 而非 graph-centric。

STORM 通过多视角问答把研究前置为 knowledge curation，再生成 outline 与文章。它是非常清晰的 workflow；WebThinker 则让研究与写作在同一长推理轨迹中交错。

18. WebThinker vs Open Deep Research / GPT Researcher

LangChain Open Deep Research 的主抽象是 Supervisor → ConductResearch subagents → Compression → Final Report。它把探索深度主要交给 tool-calling supervisor 与多个 subagent。WebThinker 则通常由一个主 LRM 保持连续的长程 reasoning state，Deep Web Explorer 作为有内部策略的高阶工具。

GPT Researcher 更强调工程化 research pipeline：query decomposition、retriever、scraper、context compression、citation/report generation。WebThinker 更接近“训练一个会研究的模型”，而 GPT Researcher 更接近“构建一个可靠研究系统”。两者的最佳组合是：**WebThinker-style policy + GPT Researcher-style evidence pipeline**。

19. WebThinker vs Tongyi / DeerFlow

Tongyi DeepResearch 把 research policy 更彻底地内化到模型训练：Agentic Continual Pretraining、SFT、RL 与专门的 environment/data engine。相较之下，WebThinker 的训练方案更轻，核心是 trajectory preference + online DPO，并大量依赖外部 Host runtime。

DeerFlow 2.0 则站在另一端：把 research 作为 Skill，核心资产是通用 Agent Harness、middleware、sandbox、filesystem、subagent、memory 与 runtime guardrails。WebThinker 的强项是策略深度，DeerFlow 的强项是长时程工程可靠性。

统一视角

可以把 2025-2026 Deep Research 演化概括为两条路线：**Policy Scaling**（WebThinker / Tongyi：让模型更会研究）与 **Harness Scaling**（Open DR / DeerFlow：让运行时更会承载研究）。成熟系统最终会合流。

20. 可复用 Know-why / Know-how

20.1 把“Research”设计成高阶工具

不要让主 Agent 直接处理所有搜索 API 细节。可以像 Deep Web Explorer 一样，把多步 Search/Read/Navigate 封装为一个能够独立收束的子策略，并只返回 task-conditioned evidence。

20.2 决策上下文与证据上下文分离

主 Planner 需要的是 progress、coverage、gaps、outline，而不是全部网页正文。网页进入 document/evidence memory，真正写作时再做局部检索。

20.3 Search 与 Click/Read 分开

搜索结果页只是候选发现层。真正的证据必须经过页面读取与 query-conditioned extraction。

20.4 写作也是 Agent Action

长报告不要等 Research 全结束后一次性生成。允许模型在“证据足够”时写局部章节，后续通过 Check/Edit 迭代。

20.5 强模型做策略，辅助模型做文本操作

复杂推理模型决定什么时候搜、点、写、改；便宜模型负责 intent extraction、page read、section rewrite，可显著降低成本。

20.6 训练评价必须包含效率

同样正确的 trajectory 中，搜索更少、推理更短的策略更适合生产。

20.7 在线数据优于静态模仿

当 policy 改善后，它遇到的新状态分布也会变化，因此重新采样轨迹再优化，比只对旧轨迹做离线偏好训练更合理。

20.8 Runtime 必须确定性兜底

LLM 决定 action；重复检测、预算、cache、timeout、解析、权限和失败恢复都应该由代码强制。

21. 从零实现 WebThinker-style MVP

建议不要一开始就训练 RL。先把结构跑通，建立可观测的 trajectory，再决定是否需要 policy optimization。

阶段	最小实现	验收指标
1. Host Loop	解析 search marker → Search API → observation injection	格式成功率、loop 可终止
2. Deep Explorer	search + click/read + duplicate guard + budget	找到深层证据、无无限循环
3. Evidence Memory	URL/title/content/source metadata + local retrieval	可追踪来源、写作无需全量上下文
4. Report Tools	draft/check/edit 三动作	章节可增量生成与修订
5. Eval	GAIA-like QA + report rubric + citation checks	质量/成本/工具数/时延可量化
6. Trajectory Dataset	保存每步 thought/action/observation/result	可重放、可比较
7. Preference Optimization	正确性 → 工具数 → 推理长度分层排序	同等质量下成本下降

```
while not done:
    token_stream = LRM(state_summary, available_tools)
    action = parse_action(token_stream)
    if action == SEARCH:
        evidence = deep_explorer(action.query)
        evidence_memory.add(evidence)
        state.observe(compress(evidence))
    elif action == DRAFT:
        docs = evidence_memory.retrieve(action.section)
        report.write(aux_writer(docs, action.instruction))
    elif action == CHECK:
        state.observe(report.outline_summary())
    elif action == EDIT:
        report = aux_editor(report, action.instruction)
    else:
        done = True
```

22. 面向 2026 的下一代改进

WebThinker 已经证明 reasoning-native search 是有效方向，但下一代系统需要进一步把“会搜索”升级为“会管理证据、会使用更多工具、会在长时程中稳定工作”。

- **Evidence Graph**: 每个 claim 绑定 supporting URLs / passages / timestamps / source quality，而不是只保存自由文本 document memory。
- **Hybrid Retrieval Memory**: BM25 + embedding + reranker + domain/source diversity，避免 Top-3 被同源页面占满。
- **Adaptive Stop Policy**: 依据 coverage、conflict、marginal information gain、预算和 confidence 停止，而不是固定搜索次数。
- **Tool Discovery / Tool Scaling**: 沿团队后续 DeepAgent 路线，让模型在大量工具中先发现/选择所需工具，而不是把所有 schema 常驻上下文。
- **Memory Folding**: 把长 trajectory 的已完成研究阶段折叠成结构化 state，释放推理上下文。
- **Citation Verifier**: 报告写完后逐 claim 检查引用存在、来源支持、引用位置正确。
- **Security Boundary**: 网页文本默认是不可信输入，和 system/tool instructions 强隔离；对页面内 prompt injection 做过滤与行为约束。
- **Policy + Harness Co-design**: 模型学习研究策略，Runtime 提供 sandbox、budget、permissions、observability、recovery。

最终架构公式

Deep Research $\approx$ Reasoning Policy $\times$ Deep Exploration Tool $\times$ Evidence Memory $\times$ Artifact State $\times$ Tool-efficient Training $\times$ Runtime Guardrails。WebThinker 最大贡献，是把前两项从 Workflow 规则变成了模型可以连续决策的策略空间。

23. 来源与证据等级

本报告区分三类证据：A = 论文 / 官方源码直接证据；B = 官方 README / 项目元数据；C = 本报告基于多系统的工程归纳与推断。

等级	来源	用于支持
A	NeurIPS 2025 paper: WebThinker: Empowering Large Reasoning Models with Deep Research Capability	方法、公式、实验、消融、训练设置、局限
A	RUC-NLPIR/WebThinker: scripts/run_web_thinker.py	Problem Solving tool loop、cache、预算、Explorer 实现
A	RUC-NLPIR/WebThinker: scripts/run_web_thinker_report.py	Report tools、BM25 document memory、article/check/edit 状态
B	RUC-NLPIR/WebThinker README	模型集合、Serper 支持、运行方式、Deep Research Agent Family
B	GitHub repository metadata, checked 2026-08-13	项目活跃状态、license、stars/forks/open issues
C	本报告跨项目比较与工程抽象	Policy Scaling vs Harness Scaling、生产化建议、统一架构公式

主要官方地址（供人工复核）：

https://proceedings.neurips.cc/paper_files/paper/2025/file/ae03bdef276132fae089692445725635-Paper-Conference.pdf

<https://arxiv.org/abs/2504.21776>

<https://github.com/RUC-NLPIR/WebThinker>

https://github.com/RUC-NLPIR/WebThinker/blob/main/scripts/run_web_thinker.py

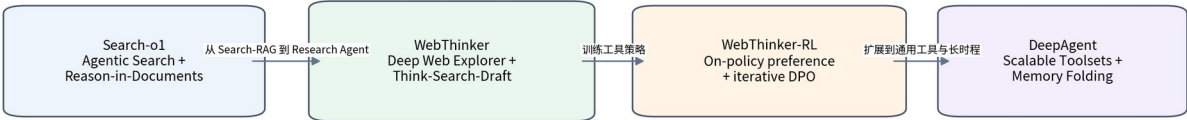
https://github.com/RUC-NLPIR/WebThinker/blob/main/scripts/run_web_thinker_report.py

当前项目状态（2026-08-13 检查）

官方仓库创建于 2025-03-28；GitHub 元数据显示 2026-08-10 仍有页面更新，最近代码 push 为 2025-12-08；MIT License；约 1.46k stars、142 forks、11 open issues。论文已被 NeurIPS 2025 接收。当前 README 还把 WebThinker 放在 Search-o1 与后续 DeepAgent 的研究演化链中。

结论：WebThinker 真正值得带走的是什么？

RUC-NLPIR Deep Research 演化：Search-o1 → WebThinker → DeepAgent



收束图 从 Search-o1 到 WebThinker：研究能力从“检索增强”走向“推理原生的研究动作”

最终判断

WebThinker 的核心不是多一个 Search Tool，而是把 Research 变成 reasoning policy 的动作空间；下一代系统应进一步把 learned research policy 与 Evidence Graph、citation verification、sandbox、budget、permissions 和 observability 结合。

最可迁移的三个架构抽象是：高阶 Research Tool 可以拥有自己的内部 agent loop；Evidence State 应外部化并按任务压缩回主推理链；报告应成为可 Draft / Check / Edit 的运行期 artifact。这三点共同把 Deep Research 从“固定搜索 workflow”推进为可持续决策的长程 Agent。

但 Agent Model 不能取代 Agent Runtime。稳定生产仍需要确定性的预算、工具协议、证据治理、安全隔离与失败恢复。因此更值得构建的不是单独复制 WebThinker，而是把它的 learned research policy 与成熟 Harness 的 deterministic guardrails 结合起来。